mini-projet – Mini-Shell en C

Implémentation d'un interpréteur de commandes Unix

Système – ET3

Durée : 8h En Binôme

Présentation

Dans ce mini-projet, vous allez implémenter **minishell**, un interpréteur de commandes Unix simplifié. À la fin des 8 heures, votre shell sera capable d'exécuter des commandes, de gérer les processus en foreground et background, et de répondre correctement aux signaux clavier (Ctrl-C, Ctrl-Z).

Le projet est découpé en **5 parties progressives**. Chaque partie s'appuie sur la précédente. Lisez l'intégralité d'une partie avant de commencer à coder.

Planning indicatif

Partie	Contenu	Durée	Cumulé
1	Squelette & boucle REPL	45 min	0h45
2	Exécution de commandes (fork/exec)	1h15	2h00
3	Redirections & pipes	1h45	3h45
4	Signaux & jobs background	2h00	5h45
5	Job control complet (fg/bg/jobs)	2h00	7h45
	Finalisation & démo	15 min	8h00

Rendu

— Un fichier `minishell.c` compilable avec `gcc -Wall -o minishell minishell.c`
— Un court `README.txt` listant ce qui fonctionne et ce qui ne fonctionne pas
— Dépôt sur le serveur de rendu avant la fin de la séance

Le code squelette fourni (`minishell_skeleton.c`) contient les includes, les structures de données, et les prototypes. Complétez les blocs marqués `/* TODO */`.

1 Squelette & boucle REPL ~45 min

Objectifs : comprendre la structure globale, implémenter la boucle REPL, parser une ligne de commande.

1.1 La boucle principale

Un shell est fondamentalement une boucle infinie : il affiche une invite (*prompt*), lit une ligne saisie par l'utilisateur, l'analyse, l'exécute, puis recommence.

Étape A – Compléter la boucle REPL dans `main()`.

```
while (1) {
    /* TODO 1a : afficher le prompt "minishell> "
       Utiliser fflush(stdout) pour forcer l'affichage */

    /* TODO 1b : lire une ligne avec fgets() */

    /* TODO 1c : si fgets() retourne NULL (EOF / Ctrl-D), quitter */

    /* TODO 1d : supprimer le '\n' final */

    int argc = parse_line(line, argv, MAX_ARGS); /* fourni */
    if (argc == 0) continue;

    /* TODO 1e : si argv[0] == "exit", quitter proprement */

    execute_command(argc, argv);
}
```

1.2 Le parseur

La fonction `parse_line()` est fournie dans le squelette. Elle découpe la ligne en tokens séparés par des espaces et les range dans `argv[]`. Lisez-la avant de continuer.

Étape B – Tester le parseur. Compilez et vérifiez que :

- `echo hello world` produit `argv[0]=echo`, `argv[1]=hello`, etc.
- Une ligne vide ne provoque pas de crash.
- `exit` termine le processus.

Note : à ce stade `execute_command()` est un stub vide.

2 Exécution de commandes ~1h15

Objectifs : créer un processus fils avec `fork()`, remplacer son image avec `execvp()`, attendre sa fin avec `waitpid()`.

2.1 fork / exec / wait

Pour exécuter une commande externe, le shell suit toujours le même schéma : fork un fils, le fils appelle exec pour se transformer en la commande souhaitée, le père attend la fin du fils.

```
pid_t pid = fork();
if (pid == 0) {
    execvp(argv[0], argv);
    perror(argv[0]);
    exit(EXIT_FAILURE);
} else if (pid > 0) {
    int status;
    waitpid(pid, &status, 0);
} else {
    perror("fork");
}
```

Étape A – Implémenter `execute_command()`.

- Complétez la fonction avec le schéma ci-dessus.
- Testez avec `ls -l`, `echo bonjour`, `pwd`.
- Gérez proprement le cas d'une commande inexistante.

2.2 Commandes internes (builtins)

Certaines commandes ne peuvent pas s'exécuter dans un fils car elles doivent modifier l'état du shell lui-même. C'est le cas de `cd` : changer de répertoire dans un fils n'affecte pas le père.

Étape B – Implémenter le builtin `cd`.

- Avant le fork, testez si `argv[0]` vaut `"cd"`.
- Si oui, appelez `chdir(argv[1])` directement (sans fork) et retournez.
- Gérez `cd` sans argument en allant dans `$HOME`.
- Testez avec `cd /tmp` puis `pwd`.

Fonctions utiles : `fork(2)`, `execvp(3)`, `waitpid(2)`, `chdir(2)`, `getenv(3)`.

3 Redirections & Pipes ~1h45

Objectifs : rediriger stdin/stdout avec `open()` + `dup2()`, chaîner des commandes avec `pipe()`.

3.1 Redirections

Les redirections connectent les entrées/sorties d'une commande à des fichiers. Le shell détecte les tokens `>`, `»`, `<` dans la ligne de commande.

```
/* Redirection de stdout vers un fichier */
int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fd, STDOUT_FILENO);
close(fd);
```

Étape A – Détecter et appliquer les redirections.

- Modifiez `parse_line()` pour détecter `>`, `»`, `<` et stocker les noms de fichiers associés.
- Dans le fils, appliquez les redirections avec `dup2()` avant `execvp()`.
- Testez : `ls -l > /tmp/out.txt` puis `cat /tmp/out.txt`.
- Testez : `cat < /etc/hostname`.

3.2 Pipes

Un pipe connecte la stdout d'une commande à la stdin de la suivante. Votre shell doit gérer au moins un pipe.

```
int pipefd[2];
pipe(pipefd);  /* pipefd[0] = lecture, pipefd[1] = ecriture */

/* Fils gauche */
dup2(pipefd[1], STDOUT_FILENO);
close(pipefd[0]); close(pipefd[1]);
execvp(...);

/* Fils droit */
dup2(pipefd[0], STDIN_FILENO);
close(pipefd[0]); close(pipefd[1]);
execvp(...);

/* Pere : fermer les deux extremites puis waitpid x2 */
```

Étape B – Implémenter `execute_pipe()`.

- Créez une fonction `execute_pipe(cmd_left, cmd_right)`.
- Détectez le token `|` dans la ligne parsée et appelez cette fonction.
- Testez : `ls | grep .c` et `cat /etc/passwd | wc -l`.
- N'oubliez pas de fermer les fd inutiles dans chaque fils ET dans le père.

Fonctions utiles : `open(2)`, `dup2(2)`, `pipe(2)`, `close(2)`.

4 Signaux & processus background ~2h00

Objectifs : initialiser le shell comme chef de groupe, lancer des jobs en background avec &, gérer SIGINT et SIGTSTP.

4.1 Initialisation du shell

Un shell interactif doit prendre le contrôle du terminal au démarrage. Sans cela, les signaux clavier seraient envoyés à tous les processus du groupe.

```
void init_shell(void) {
    while (tcgetpgrp(STDIN_FILENO) != (shell_pgid = getpgrp()))
        kill(-shell_pgid, SIGTTIN);

    signal(SIGINT,  SIG_IGN);
    signal(SIGQUIT, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTTOU, SIG_IGN);

    shell_pgid = getpid();
    setpgid(shell_pgid, shell_pgid);
    tcsetpgrp(STDIN_FILENO, shell_pgid);

    /* TODO : sauvegarder les attributs du terminal avec tcgetattr() */
}
```

Étape A – Compléter `init_shell()`.

- Remplissez les `TODO` du squelette.
- Sauvegardez les attributs du terminal avec `tcgetattr()`.
- Appelez `init_shell()` au début de `main()`.

4.2 Jobs en foreground et background

Quand une commande se termine par `&`, le shell n'attend pas sa fin et réaffiche immédiatement le prompt. En foreground, le shell cède le terminal au fils.

Étape B – Détecter `&` et lancer en background.

```
/* Dans le fils */
setpgid(0, 0);
if (foreground)
    tcsetpgrp(shell_terminal, getpgrp());
signal(SIGINT,  SIG_DFL);
signal(SIGTSTP, SIG_DFL);
execvp(argv[0], argv);

/* Dans le pere */
setpgid(pid, pid);
if (foreground) {
    wait_for_job(pid);
    tcsetpgrp(shell_terminal, shell_pgid);
} else {
    printf("[%d] %d\n", next_job_id - 1, pid);
}
```

- Détectez si le dernier token est `&` et retirez-le de `argv[]`.

— Créez `launch_job(argc, argv, foreground)` avec le schéma ci-dessus.
— Testez : `sleep 5 &` – le prompt doit réapparaître immédiatement.

4.3 Ctrl-C et Ctrl-Z

Étape C – Vérifier le comportement des signaux.

— `sleep 10` + Ctrl-C : `sleep` doit mourir, le shell doit rester.
— `sleep 10` + Ctrl-Z : `sleep` doit être suspendu, le shell reprend.
— Le shell lui-même ne doit pas mourir dans les deux cas.

Fonctions utiles : `setpgid(2)`, `tcsetpgrp(3)`, `tcgetattr(3)`, `signal(2)`, `kill(2)`.

5 Job control complet ~2h00

Objectifs : maintenir une liste de jobs, implémenter `fg`, `bg` et `jobs`, éviter les processus zombies.

5.1 Liste des jobs

```
typedef enum { RUNNING, STOPPED, DONE } job_status_t;

typedef struct job {
    int           job_id;
    pid_t         pgid;
    char          command[256];
    job_status_t  status;
    struct job   *next;
} job_t;

job_t *job_list = NULL;
```

Étape A – Gérer la liste de jobs.

- Complétez `add_job()` et `remove_job()` dans le squelette.
- Appelez `add_job()` après chaque `fork()` réussi.

5.2 Builtin `jobs`

Étape B – Implémenter la commande `jobs`.

```
[1]  Running    sleep 30 &
[2]  Stopped    vim README.txt
[3]  Done       ls -l
```

Appelez `update_job_statuses()` avant d'afficher, pour rafraîchir les statuts via `waitpid(WNOHANG|WUNTRACED)`.

5.3 Builtins `fg` et `bg`

Étape C – Implémenter `fg` et `bg`. Syntaxe : `fg %1` ou `fg` (dernier job).

- **fg** : envoyer `SIGCONT` au groupe du job, lui rendre le terminal, puis appeler `wait_for_job()`.
- **bg** : envoyer `SIGCONT` au groupe du job sans lui rendre le terminal.
- Testez : `sleep 20` + Ctrl-Z, puis `bg`, puis `fg`.

5.4 Éviter les processus zombies

Sans collecte des zombies, les jobs terminés restent dans la table des processus.

Étape D – Nettoyage automatique.

- Appelez `update_job_statuses()` au début de chaque itération de la boucle principale, avant d'afficher le prompt.
- Cette fonction appelle `waitpid(-1, &status, WNOHANG|WUNTRACED)` en boucle et met à jour `job_list`.
- Affichez un message quand un job background se termine : `[1] Done  sleep 5`.
- Vérifiez avec `ps aux | grep defunct` qu'il n'y a plus de zombies.

Fonctions utiles : `waitpid(WNOHANG|WUNTRACED)`, `WIFEXITED`, `WIFSTOPPED`, `kill(-pgid, SIGCONT)`.